

Theoretical Computer Science Exam, February 18, 2020

The exam consists of **4 exercises**. Available time: 1 hr 50 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME L.NAME MATRICOLA

I have passed the midterm exam and I intend to skip the first part (Ex.1-2) (select one): **YES NO**

Exercise 1 (8 pts)

- 1) Let L be any regular language on alphabet A , and let a be any symbol of A . Then consider the language L' that includes exactly (i.e., all and only) the strings of L that include at least one occurrence of symbol a . Is language L' regular? Suitably motivate your answer.
- 2) Consider now the context-free language $L_1 = \{ (a|b)^n c^n \mid n > 0 \}$.
 - a) Design a PDA that accepts L_1 .
 - b) Consider next the language L_1' that includes exactly (i.e., all and only) the strings of L_1 that include at least one occurrence of symbol a . Is language L_1' regular? Suitably motivate your answer and design the minimum power automaton that accepts L_1' .
 - c) (Optional) Generalize your previous answer by considering any language $L_i = L_c \cap L_r$ equal to the intersection between a context-free language L_c and a regular language L_r : what is the smallest language family to which L_i belongs? Suitably motivate your answer.

Exercise 2 (8 pts)

Consider a subprogram having four parameters: an array of integers in , its size nIn , an array of integers out , its size $nOut$. The input parameters are: in , nIn , and $nOut$, whereas the array out is an output parameter. The size (i.e., number of components) of the two arrays can be assumed to have the same value, greater than 2, and all the elements of array in are assumed to be strictly positive.

After execution of the subprogram all even-valued elements of in must be stored at some odd-index position (index 1, 3, etc.) of array out , and all odd-valued elements of in must be stored at some even-index position (index 0, 2, etc.) of array out . Notice that indices of array positions are assumed to start from 0. For instance, the specification is satisfied by the following values: $in=[2, 5, 3, 4]$, $out=[3, 4, 5, 2]$; it is *not* satisfied by the following ones: $in=[2, 5, 3, 4]$, $out=[3, 5, 4, 2]$. Answer the following questions:

1. write pre- and post-conditions specifying the requirements of the above described procedure;
2. is the specification always satisfiable? If not, say in which cases it is not satisfiable;
3. modify the precondition in such a way that the postcondition can always be made true by a suitable correct implementation.

Exercise 3 (8 pts)

Is it decidable whether a generic sentence of the Monadic First Order logic on strings is satisfiable or not? Explain clearly your answer.

Exercise 4 (8 pts)

Consider the following language

$$L = \{ a^{h_1} b^{k_1} a^{h_2} b^{k_2} \dots a^{h_n} b^{k_n} \mid n > 0 \wedge \forall i (1 \leq i \leq n \rightarrow h_i > k_i) \wedge \forall i (1 \leq i < n \rightarrow k_i > h_{i+1}) \}$$

The language strings are alternated sequences of letters a and b ; the number of sequences is positive and even; each sequence of equal letters (be it made of a 's or b 's) is shorter than all the preceding ones and longer than all the following ones. Thus, for instance, $aaaaabbbaab \in L$, $aaaabbbb \in L$, while $\varepsilon \notin L$, $aaabbbb \notin L$, $aaaabba \notin L$, $aaaabbbbaaab \notin L$.

1. Design, preferably in full detail, or at least describe in a clear and sufficiently detailed way, a 1-tape Turing machine (i.e., having one memory tape) that accepts language L and determine its time and space complexity class. Also determine the worst case and best case space complexity function in case the input string belongs to the language L . Provide suitable justifications for your answers.
2. Describe, in a clear and sufficiently detailed way, a RAM machine accepting the same language L , and determine its time and space complexity class, both adopting the uniform and the logarithmic cost criterion. Provide suitable justifications for your answers.

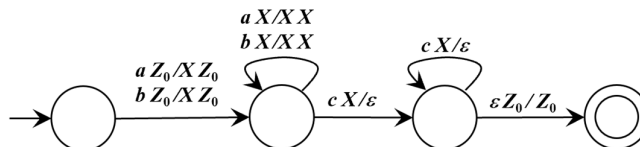
Solutions

Exercise 1

1) Let L'' be the language of all strings over alphabet A containing at least one a . This language is clearly regular. The language L' is equal to the intersection of L and L'' , and these are both regular. Then L' is also regular, because the family of regular languages is closed under intersection.

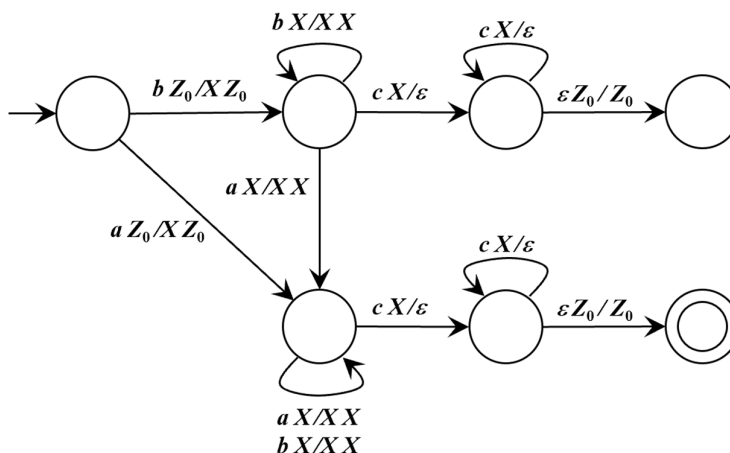
2) Consider now the context-free language $L_1 = \{ (a \mid b)^n c^n \mid n > 0 \}$.

a) Design a deterministic PDA that accepts L_1 .



b) Consider next the language L_1' that includes exactly (i.e., all and only) the strings of L_1 that include at least one occurrence of symbol a . Is language L_1' regular? Suitably motivate your answer by designing the minimum power automaton that accepts L_1' .

Accepting L_1' still requires the ability to count the number of a 's and b 's and check that their total number equals the number of c 's. Therefore the language is not regular and a PDA is needed. It can be obtained as a variant of the PDA accepting L_1 by adding states that allow the automaton to “remember” that an a was read from the input.



c) L_i is a context-free language. Informally, the PDA that accepts it can be obtained from the one that accepts L_c by a Cartesian product of the states of L_c and those of L_r ; the management of the stack is unchanged.

Exercise 2

1.

pre: $nIn > 2 \wedge nIn = nOut \wedge \forall i (0 \leq i < nIn \rightarrow in[i] > 0)$

post: let $even(i) \leftrightarrow \exists k(i=2k)$ and $odd(i) \leftrightarrow \exists k(i=2k+1)$; then the postcondition is

$\forall i ((0 \leq i < nIn \wedge even(in[i]) \rightarrow \exists j (0 \leq j < nOut \wedge odd(j) \wedge in[i] = out[j])) \wedge$
 $(0 \leq i < nIn \wedge odd(in[i]) \rightarrow \exists j (0 \leq j < nOut \wedge even(j) \wedge in[i] = out[j]))))$
 $)$

2. Given the stated precondition, the postcondition is not satisfiable in case the number of even-valued (or, respectively, odd-valued) elements of array *in* exceeds the number of odd index positions (resp., even index positions) in array *out*; for instance if $in = [2, 8, 6, 4]$
3. If the size of the *out* array parameter is at least twice that of the array *in*, i.e., if $nOut \geq 2 \cdot nIn$, then all elements of the array *in* can be stored in suitable positions of the array *out* in all cases.

Exercise 3

The satisfiability of an MFO sentence is decidable, because there is an algorithm that, for any given MFO sentence *F*, builds an equivalent FSA, that is, an FSA that accepts exactly the set of strings that satisfy the sentence *F*. Therefore the sentence *F* is satisfiable if and only if the FSA equivalent to it accepts a non-empty language; this latter problem is notoriously decidable.

Exercise 4

1. The TM first reads the first sequence of *a*'s and stores in the tape, in unary using e.g. a '+' symbol, its length, i.e., the number h_1 . Then it reads the sequence of *b*'s while it moves leftward on the memory tape; if it reaches the end of the sequence of *b*'s before the first '+' on the memory tape then it changes the current '+' into a '*', otherwise it rejects the string. Then it continues in a similar way reading the sequences of *a*'s and *b*'s, alternating scans of the memory tape from left to right (while reading *a*'s) and from right to left (while reading *b*'s). It accepts the input string if and only if the last sequence of letters is composed of *b* letters.

Let $n = |x|$ be the input string length; then the time complexity is $\Theta(n)$ and also the space complexity is $\Theta(n)$; in the worst case, i.e., that of a string of type a^n , a string of n symbol '+' is written on the memory tape.

In case the input string is accepted the worst space complexity is $S(n) = n - 1$ (string of type $a^{n-1}b$) and the best case complexity is $S(n) = \sqrt{n}$ (string of type $a^k b^{k-1} a^{k-2} b^{k-3} \dots a^2 b$).

2. A RAM machine reads all sequences and counts their elements, checking that the length of each sequence is less than that of the previous one. With the uniform cost criterion the space complexity is $\Theta(1)$ (a constant amount of memory is needed), while the time complexity is $\Theta(n)$. With the logarithmic cost criterion the space complexity is $\Theta(\log n)$ (for storing a fixed number of counters) and the time complexity is $\Theta(n \log n)$ (for computing the value of the largest possible counter by a sequence of n increment operations).